

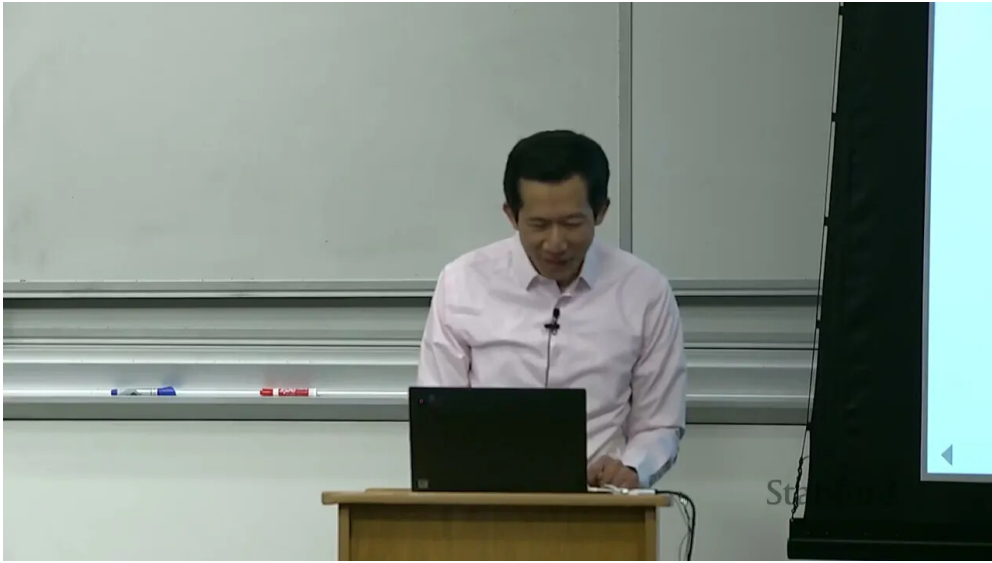
课程笔记

CS336 Lecture 8: 多 GPU 并行 (Parallelism 2)

Stanford CS336: Language Modeling from Scratch (Spring 2025)

讲师: Percy Liang, Tatsu Hashimoto 笔记: ZCode

2026-07-14



视频作者/频道: Stanford CS336

发布日期: 2025 Spring

视频时长: 75 分钟

视频链接: <https://www.youtube.com/watch?v=LHpr5ytssLo>

Contents

1	从单 GPU 到多 GPU: 本周的全景	2
1.1	系统讲座的第二周	2
1.2	硬件拓扑: 节点、GPU、SM 与互连	2
1.3	三种切分维度的全景	2
2	集合通信原语: 并行的字母表	3
2.1	术语: world size 与 rank	3
2.2	七个原语的总览	3
2.3	最关键的一组等价关系	7
2.4	PyTorch 中的 API	7
3	用数值演示验证等价关系	8
4	通信成本与带宽基准测试	10
4.1	ring all-reduce 的实现	10
4.2	如何测量带宽	11
4.3	为什么 all-reduce 有 2x 因子	12
5	数据并行 (Data Parallelism / DDP)	13
5.1	沿 batch 维切分	13
5.2	DDP 的实现: 在 SGD 里“注入”一行 all-reduce	14
5.3	DDP 的关键性质	15
6	张量并行 (Tensor Parallelism, Megatron 风格)	16
6.1	沿 hidden dim 切分模型	16
6.2	前向传播: all-gather 拼回激活	16
6.3	通信代价与适用场景	18
7	流水线并行 (Pipeline Parallelism)	18
7.1	按层切分模型	18
7.2	气泡 (bubbles): 朴素实现的代价	19
7.3	send / recv: 点对点通信	20
7.4	通信与计算的重叠	21
8	3D 并行、FSDP 与声明式分片	22
8.1	三种并行的组合: 3D Parallelism	22
8.2	FSDP 与 ZeRO: 参数分片	23
8.3	PyTorch 手动实现 vs Jax 声明式分片	23
9	总结与延伸	24
9.1	核心论点	24
9.2	课堂问答: 硬件演进与持续训练	24
9.3	三种并行的对比总结	24
9.4	下节预告与配套阅读	25

1 从单 GPU 到多 GPU：本周的全景

1.1 系统讲座的第二周

这是 systems 模块的第二周。上一讲 (Parallelism 1) 聚焦于单块 GPU 内部的并行——如何通过 kernel fusion、tiling、occupancy 等手段榨取一块卡的峰值算力。本讲把视角拉远，转向多 GPU 之间的并行：当一块卡的显存或算力装不下整个训练任务时，如何把工作切分到多块卡、多个节点上协同完成。

本讲的中心问题

给定一个放不进单卡的训练任务，我们该沿哪个维度切分它？切完之后，各卡之间需要交换什么数据、用哪种集合通信原语？通信和计算如何重叠？这是大规模训练工程的全部核心。

1.2 硬件拓扑：节点、GPU、SM 与互连

理解多 GPU 并行，必须先有一张清晰的硬件拓扑图。本讲开篇回顾了这张图：

- Node (节点)：一台机器，通常配 8 块 GPU。
- GPU：每块 GPU 内有大量 SM (Streaming Multiprocessor)，它们才是真正干活的计算单元。
- 内存层次：每个 SM 内有极小但极快的 L1 cache；每块 GPU 板载较大的 HBM (高带宽显存)；节点间/卡间通过高速链路互连。
- 互连：节点内 GPU 之间走 NVLink/NVSwitch (高带宽，绕开 PCIe/CPU)；节点之间走 以太网/InfiniBand。

仓库-工厂类比 (承自 L1)

内存是仓库 (存参数、激活、梯度)，计算是工厂 (SM 执行矩阵乘)。瓶颈几乎从不在工厂产能，而在搬运成本——把数据从一处搬到另一处的时间，往往远超计算本身。因此多 GPU 并行的本质是：在尽量减少数据搬运的前提下，把工作摊到多块卡上。

为什么节点内和节点间的策略不同？

关键在于带宽落差。节点内 8 块 GPU 通过 NVLink/NVSwitch 直连，绕开 CPU 和以太网，带宽可达数百 GB/s，且延迟极低；而节点之间走以太网或 InfiniBand，带宽低一个数量级以上。这个落差直接决定了并行策略的部署：通信密集的并行 (如张量并行) 必须留在节点内，而通信稀疏的并行 (如数据并行的 all-reduce、流水线的 send/recv) 才能跨节点。理解这一点，是后面 3D 并行拓扑设计的起点。

1.3 三种切分维度的全景

本讲核心是三种把训练任务切到多卡的方式，每种沿不同维度切：

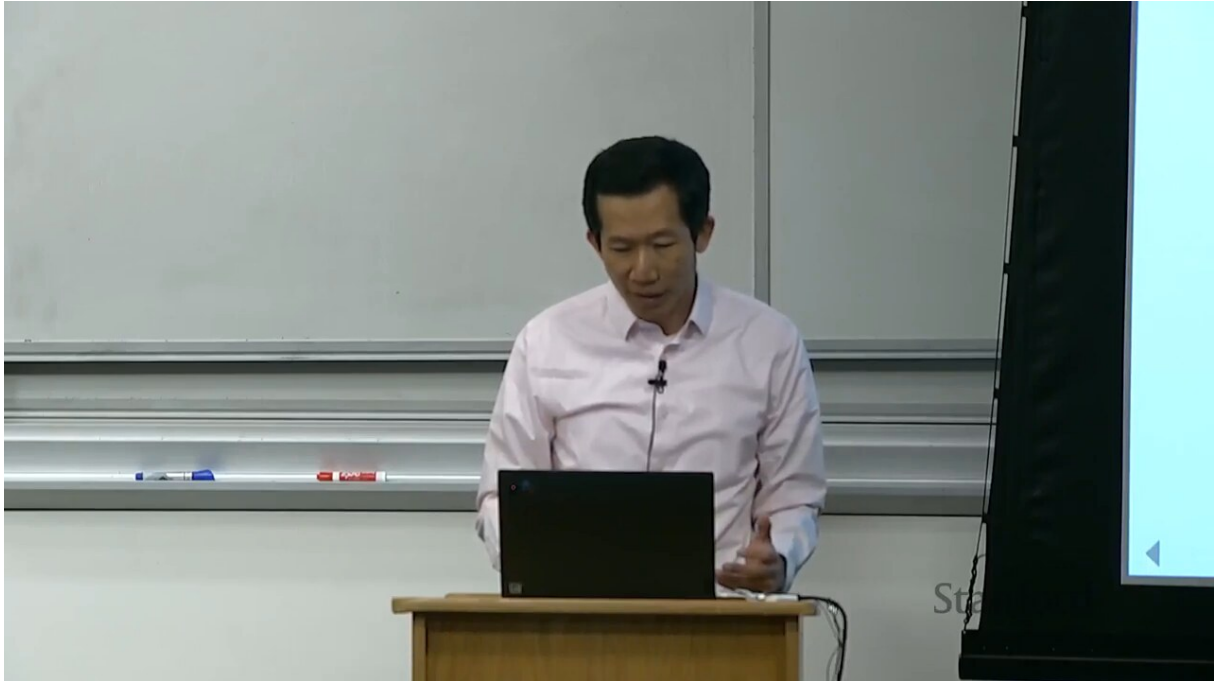


Figure 1: 三种并行维度的全景：沿 batch / 宽度 (hidden dim) / 深度 (layers) 切分¹

并行方式	切分维度	每张卡拿到什么
数据并行 (DP)	batch 维	完整模型 + 一份不同的数据子集
张量并行 (TP)	宽度 (hidden dim)	完整数据 + 每层参数的一块切片
流水线并行 (PP)	深度 (layers)	完整数据 + 连续的若干层

本讲的教学策略

Percy 用一个最朴素的 4 层 MLP 来演示三种并行。理由是：在当前规模下，MLP 的矩阵乘是 compute bottleneck（而非 attention），所以 MLP 足以代表真实训练的核心计算模式，又能把并行的机制看得一清二楚。但真要用在 Transformer 上时，代码复杂度会暴涨（见第 8 节）。

2 集合通信原语：并行的字母表

2.1 术语：world size 与 rank

在进入具体原语前，先固定术语：

两个核心术语

- **world size**：参与这次集合通信的设备总数（如 4 块 GPU）。
- **rank**：某台设备的编号（rank 0, rank 1, ..., rank $N-1$ ）。注意：这里的 rank 与线性代数里的“秩”无关，只是一个设备 ID。

2.2 七个原语的总览

集合通信（collective communication）是多 GPU 并行的“字母表”。下面七个原语经久耐用，几乎所有并行策略都由它们组合而成：

¹ 视频画面时间区间：00:01:15 – 00:01:45。

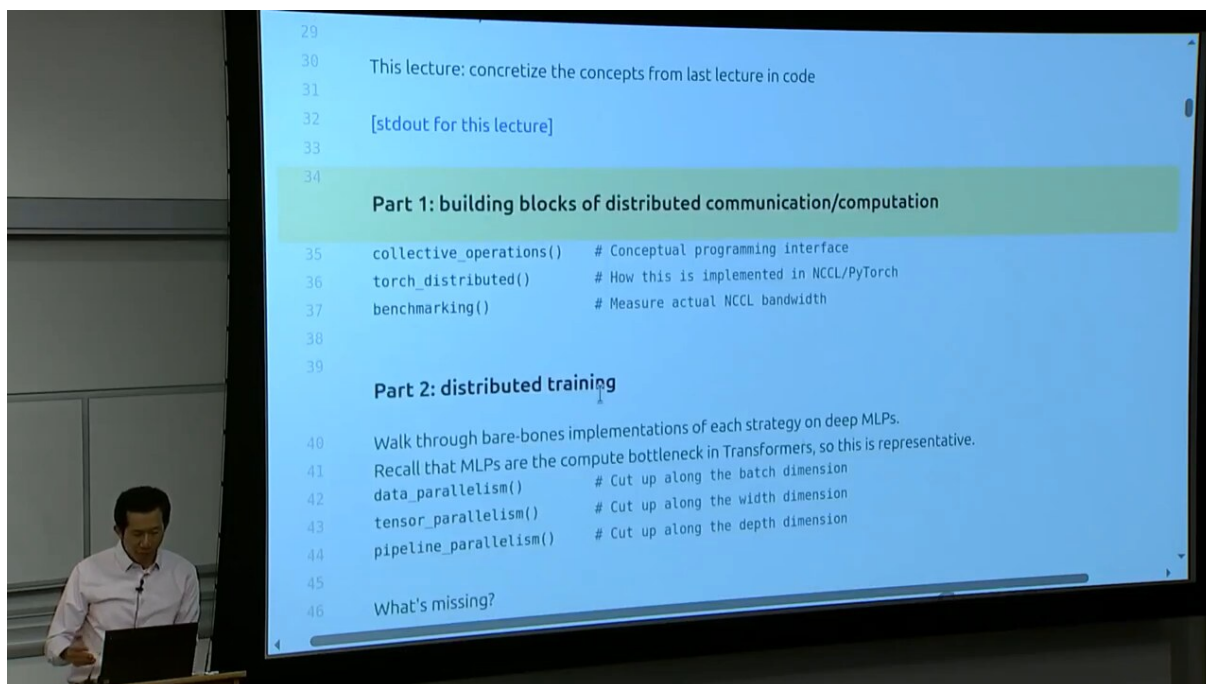


Figure 2: 七个集合通信原语总览: broadcast / scatter / gather / reduce / all-reduce / reduce-scatter / all-gather²

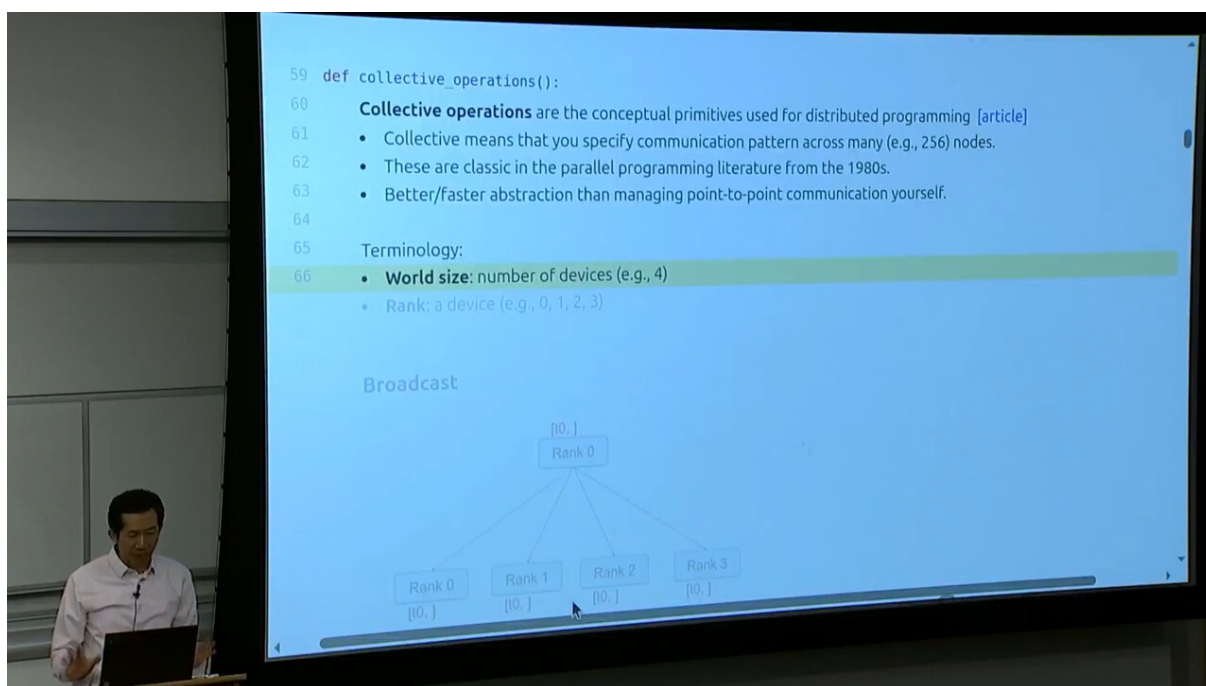


Figure 3: broadcast (同一值发全员) vs scatter (不同值分发到各 rank)³

²视频画面时间区间: 00:04:15 – 00:04:45。

³视频画面时间区间: 00:05:00 – 00:05:30。

原语	行为	说明
broadcast	一个 rank 的张量复制到所有 rank	$t_0 \rightarrow$ 全员
scatter	一个 rank 的多块张量分发到不同 rank	每个 rank 拿到不同的一块
gather	各 rank 的张量拼接到一个 rank	scatter 的逆，拼接 (concat)
reduce	各 rank 的张量按算子合并到一个 rank	类似 gather，但用 sum/min/max 而非拼接
all-reduce	reduce 到所有 rank	= reduce-scatter + all-gather
reduce-scatter	reduce 后把结果分块到各 rank	每个 rank 拿到合并结果的一块
all-gather	各 rank 的张量拼接到所有 rank	gather 的"all"版本

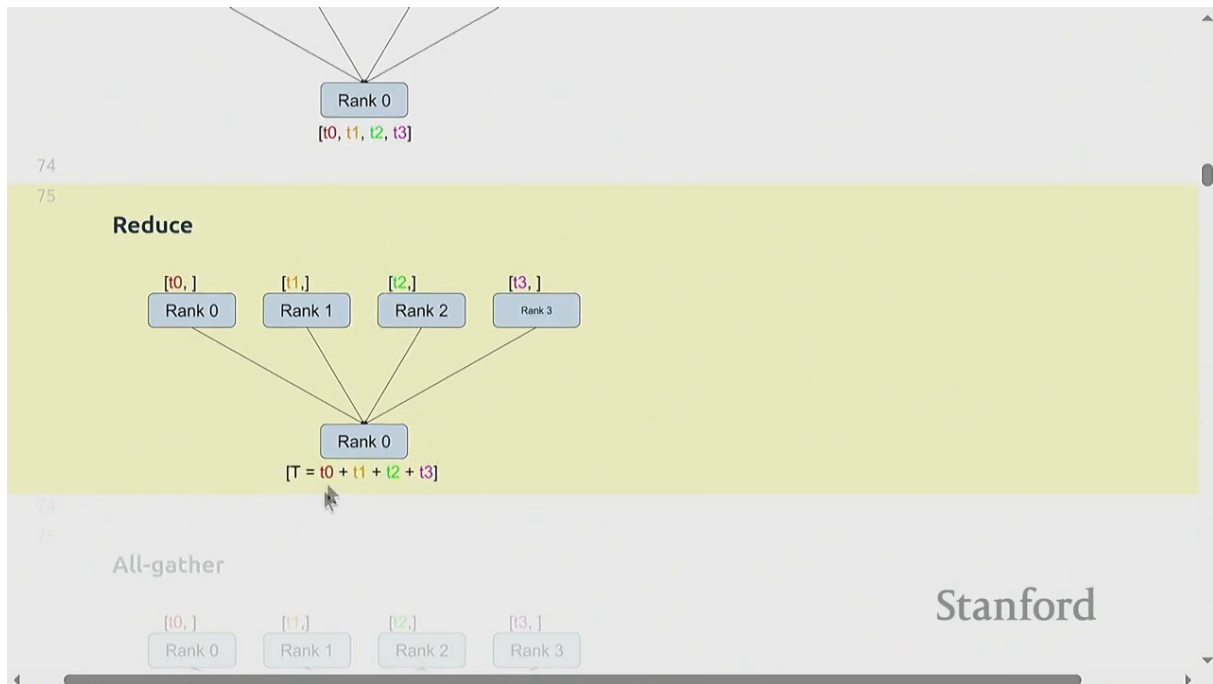


Figure 4: gather（拼接）vs reduce（用 sum 等算子合并到一个 rank）⁴

⁴视频画面时间区间：00:06:00 – 00:06:30。

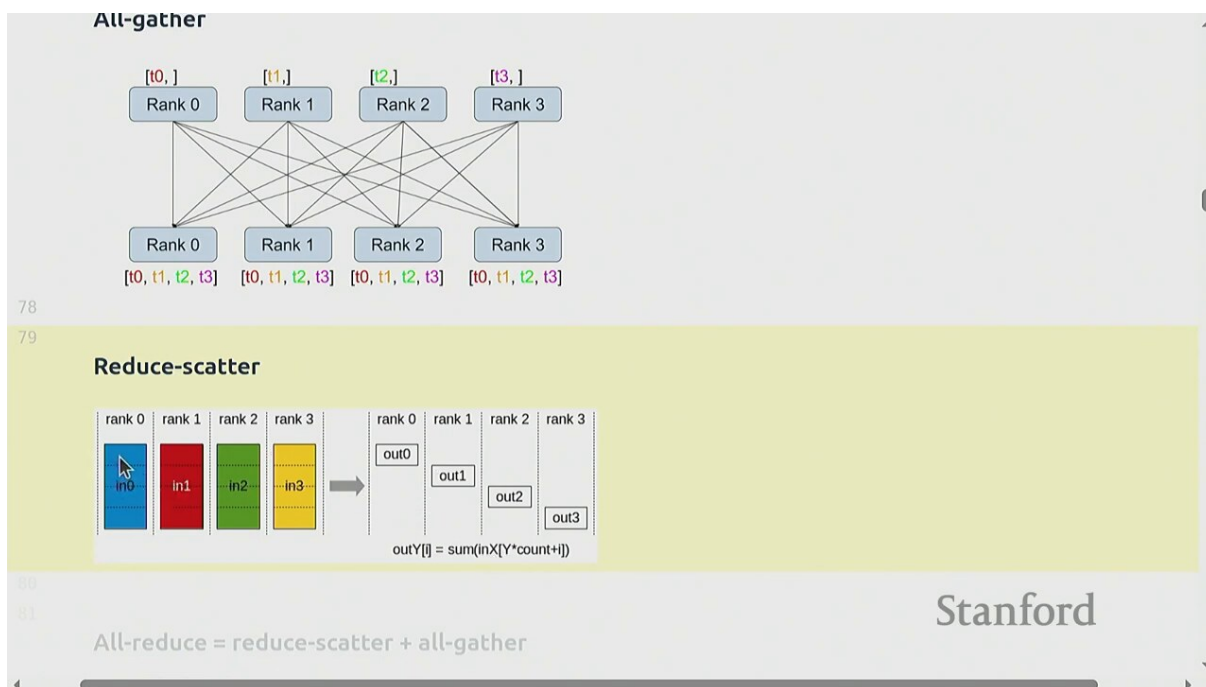


Figure 5: reduce-scatter: 先 reduce 再 scatter, 每个 rank 拿到合并结果的一块⁵

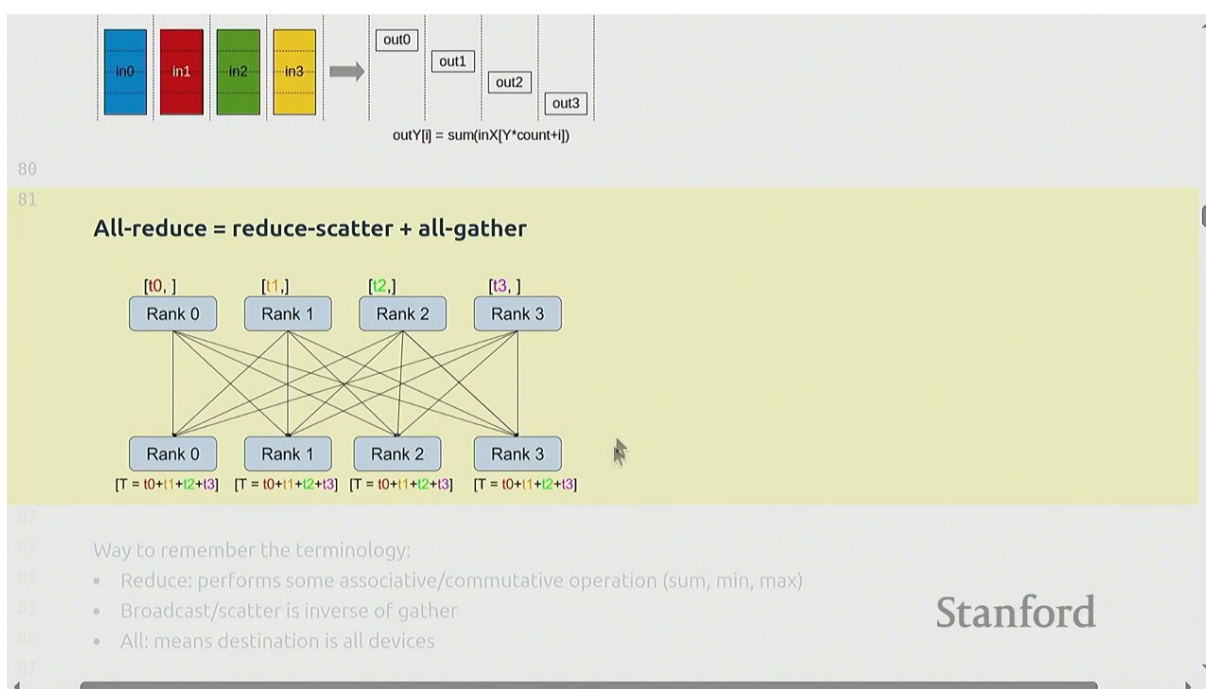


Figure 6: all-gather: 各 rank 的分块拼接后, 所有 rank 都拿到完整张量⁶

⁵视频画面时间区间: 00:06:30 – 00:07:00。

⁶视频画面时间区间: 00:07:00 – 00:07:30。

2.3 最关键的一组等价关系

80 outY[i] = sum(inX[Y*count+i])

81

All-reduce = reduce-scatter + all-gather

82

83 Way to remember the terminology:

84 • Reduce: performs some associative/commutative operation (sum, min, max)

85 • Broadcast/scatter is inverse of gather

86 • All: means destination is all devices

87

88 Stanford

89 def torch_distributed():

90

Figure 7: 原语之间的关系：reduce = reduce-scatter + all-gather；broadcast/scatter 与 gather 互逆⁷

记忆这套术语的关键，是一组分解关系。设 N 为 world size，张量 T 被均分为 N 块 $T_0, \dots, T_{N-1}$ ：

记忆口诀

- reduce = 做结合律/交换律运算 (sum / min / max / mean) 。
- scatter / broadcast 是 gather 的逆。
- all = 目的地是所有设备。

最核心的恒等式是：

$$\text{all-reduce} = \text{reduce-scatter} + \text{all-gather}$$

- reduce-scatter: 各 rank 把按位求和的结果分块，每个 rank 持有总和的一块。
- all-gather: 把这一块拼回完整张量，于是每个 rank 都拿到完整总和。

这个恒等式不仅是记忆工具，更是后面分析通信成本和 ZeRO/FSDP 设计的基石。

2.4 PyTorch 中的 API

在 PyTorch 分布式 (torch.distributed) 里，这些原语都是现成的调用：

⁷视频画面时间区间：00:07:30 – 00:08:00。

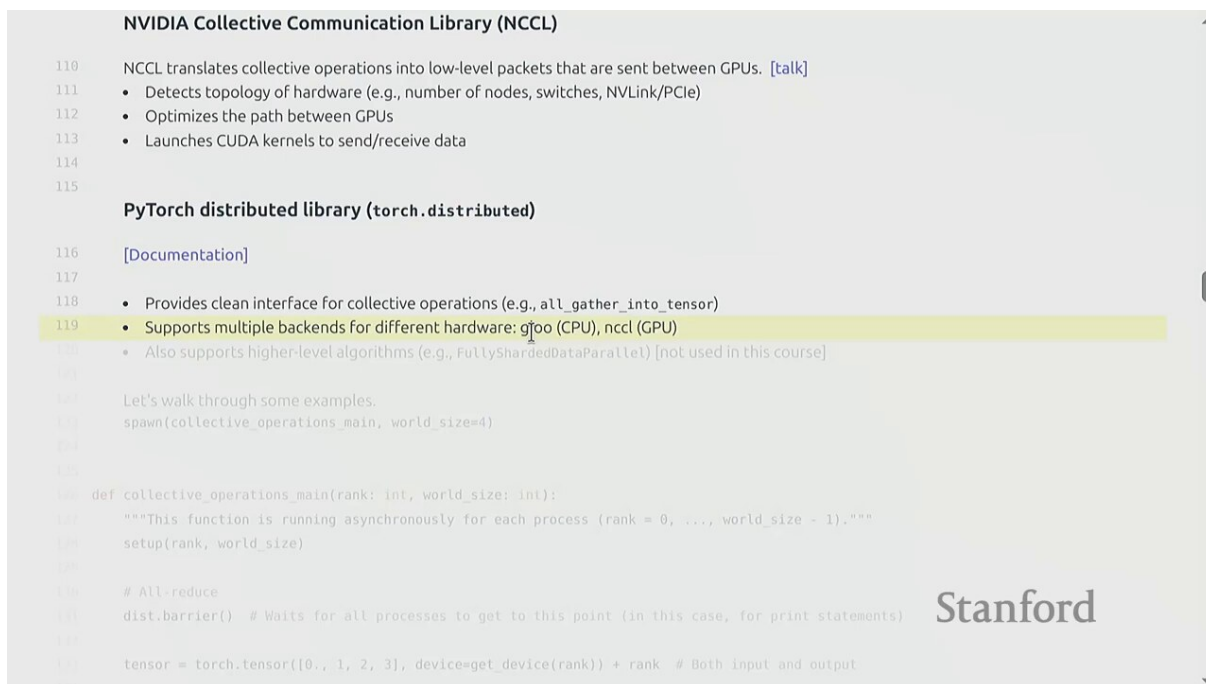


Figure 8: PyTorch 分布式 API: `dist.all_reduce` / `all_gather` / `reduce_scatter` 的用法⁸

```
1 import torch.distributed as dist
2 # all-reduce: 每个 rank 都有一个包含 4 个元素的张量
3 dist.all_reduce(param.grad, op=dist.ReduceOp.SUM)
4 # reduce-scatter: 每个 rank 都有一个包含 4 个元素的张量
5 dist.reduce_scatter(output_list, input_list)
6 # all-gather: 每个 rank 都有一个包含 4 个元素的张量
7 dist.all_gather(output_list, local_tensor)
```

3 用数值演示验证等价关系

Percy 用一个具体的数值例子，让听众“亲眼”看到 `reduce-scatter` + `all-gather` 确实等于 `all-reduce`。设 `world size = 4`，每个 `rank` 持有一个含 4 个元素的张量。

⁸视频画面时间区间：00:15:15 – 00:15:45。

```

126 def collective_operations_main(rank: int, world_size: int):
127     """This function is running asynchronously for each process (rank = 0, ..., world_size - 1)."""
128     setup(rank, world_size)
129
130     # All-reduce
131     dist.barrier() # Waits for all processes to get to this point (in this case, for print statements)
132
133     tensor = torch.tensor([0., 1, 2, 3], device=get_device(rank)) + rank # Both input and output
134
135     print(f"Rank {rank} [before all-reduce]: {tensor}", flush=True)
136     dist.all_reduce(tensor=tensor, op=dist.ReduceOp.SUM, async_op=False) # Modifies tensor in place
137     print(f"Rank {rank} [after all-reduce]: {tensor}", flush=True)
138
139     # Reduce-scatter
140     dist.barrier()
141
142     input = torch.arange(world_size, dtype=torch.float32, device=get_device(rank)) + rank # Input
143     output = torch.empty(1, device=get_device(rank)) # Allocate output
144
145     print(f"Rank {rank} [before reduce-scatter]: input = {input}, output = {output}", flush=True)
146     dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
147     print(f"Rank {rank} [after reduce-scatter]: input = {input}, output = {output}", flush=True)
148
149     # All-gather
150     dist.barrier()
151
152     input = output # Input is the output of reduce-scatter

```

Stanford

Figure 9: reduce-scatter 数值演示：4 个 rank 各持 $[0, 1, 2, 3]$ 类的块，求和后各拿一列⁹

第一步：reduce-scatter。每个 rank 的张量被分成 4 列，按列求和：

rank r 拿到第 r 列之和 $\Rightarrow$ rank₀ $\rightarrow$ 6, rank₁ $\rightarrow$ 10, rank₂ $\rightarrow$ 14, rank₃ $\rightarrow$ 18

第二步：all-gather。把这 4 个结果拼回：

```

137     print(f"Rank {rank} [after all-reduce]: {tensor}", flush=True)
138
139     # Reduce-scatter
140     dist.barrier()
141
142     input = torch.arange(world_size, dtype=torch.float32, device=get_device(rank)) + rank # Input
143     output = torch.empty(1, device=get_device(rank)) # Allocate output
144
145     print(f"Rank {rank} [before reduce-scatter]: input = {input}, output = {output}", flush=True)
146     dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
147     print(f"Rank {rank} [after reduce-scatter]: input = {input}, output = {output}", flush=True)
148
149     # All-gather
150     dist.barrier()
151
152     input = output # Input is the output of reduce-scatter
153     output = torch.empty(world_size, device=get_device(rank)) # Allocate output
154
155     print(f"Rank {rank} [before all-gather]: input = {input}, output = {output}", flush=True)
156     dist.all_gather_into_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
157     print(f"Rank {rank} [after all-gather]: input = {input}, output = {output}", flush=True)
158
159     # Indeed, all-reduce = reduce-scatter + all-gather!
160
161     cleanup()

```

Stanford

Figure 10: all-gather 演示：把 reduce-scatter 的分块输出拼回 $[6, 10, 14, 18]$ ，与 all-reduce 结果一致¹⁰

所有 rank 最终都拿到 $[6, 10, 14, 18]$

⁹视频画面时间区间：00:18:45 – 00:19:15。

¹⁰视频画面时间区间：00:21:00 – 00:21:30。

演示的结论

reduce-scatter + all-gather 产出的结果，和直接做一次 all-reduce 逐位相等。这验证了恒等式 $\text{all-reduce} = \text{reduce-scatter} + \text{all-gather}$ ，也为下一节分析通信成本提供了分解视角。

4 通信成本与带宽基准测试

4.1 ring all-reduce 的实现

```
150 dist.all_gather_into_tensor(output_tensor=output, input_tensor=input, async_op=False)
151
152 input = output # Input is the output of reduce-scatter
153 output = torch.empty(world_size, device=get_device(rank)) # Allocate output
154
155 print(f"Rank {rank} [before all-gather]: input = {input}, output = {output}", flush=True)
156 dist.all_gather_into_tensor(output_tensor=output, input_tensor=input, async_op=False)
157 print(f"Rank {rank} [after all-gather]: input = {input}, output = {output}", flush=True)
158
159 Indeed, all-reduce = reduce-scatter + all-gather!
160
161 cleanup()

30
164 def benchmarking():
    Let's see how fast communication happens (restrict to one node).

    # All-reduce
    spawn(all_reduce, world_size=4, num_elements=100 * 1024**2)

    # Reduce-scatter
    spawn(reduce_scatter, world_size=4, num_elements=100 * 1024**2)

    # References
    How to reason about operations
    Sample code

    def all_reduce(ranks: int, world_size: int, num_elements: int):
```

Figure 11: all-reduce 的 ring 实现：分步 reduce 再分步 broadcast，通信量为 $\frac{2(N-1)}{N} \cdot \text{size}$ ¹¹

实际的 all-reduce 通常用 ring 拓扑实现：把 N 个 rank 排成环，数据沿环分两阶段流动——先 reduce（每一步每个 rank 收一块、合并、再传下一站），再 broadcast（逆向把完整结果传遍全环）。

设张量大小为 S 字节，world size 为 N ，则每个 rank 在整个 ring all-reduce 中实际发出的数据量为：

$$\text{每 rank 发送量} = \frac{2(N-1)}{N} \cdot S$$

- 系数 2：来自两个阶段（reduce-scatter 阶段 + all-gather 阶段），各贡献 $\frac{N-1}{N}S$ 。
- 当 N 很大时， $\frac{2(N-1)}{N} \rightarrow 2$ ，即每 rank 接近发送 $2S$ 字节。

¹¹视频画面时间区间：00:23:45 – 00:24:15。

4.2 如何测量带宽

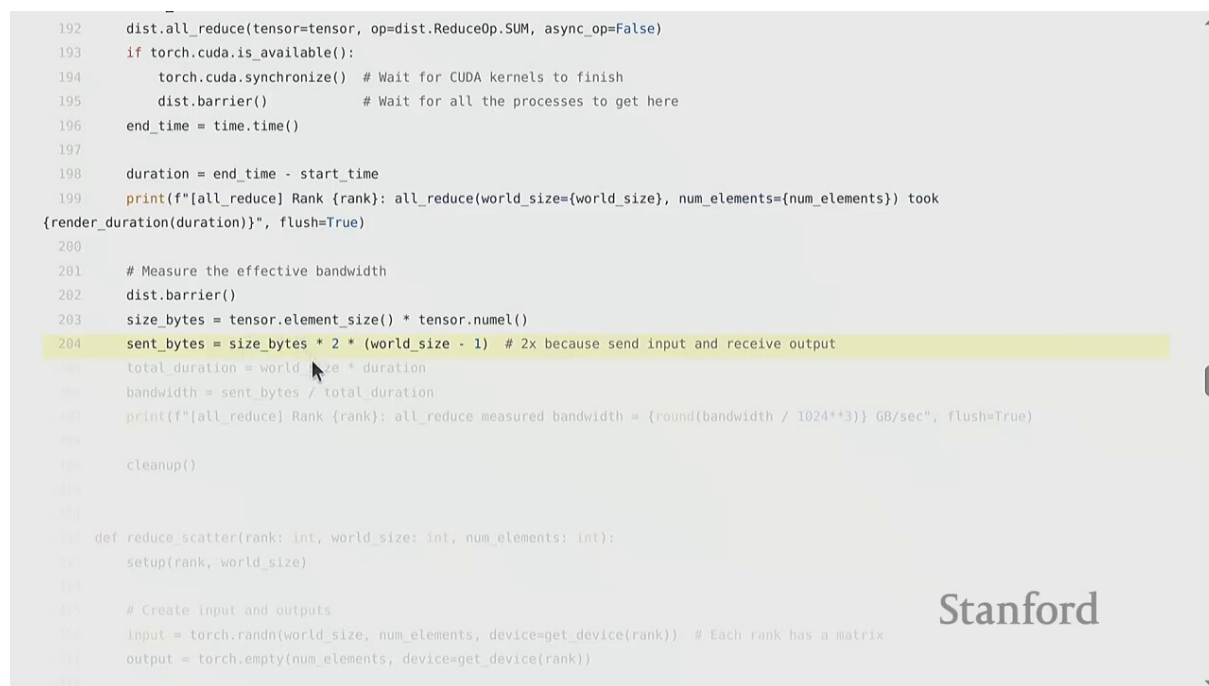


Figure 12: 带宽基准测试：测量 all-reduce 在不同张量大小下的耗时与字节传输量¹²

基准测试有严格的“清口”流程，避免把 kernel 加载、warmup 的开销算进计时：

1. **warm up**：先跑一次操作，再 **synchronize + barrier**，确保所有 kernel 已加载、缓存已就位。
2. **start clock**：记录起始时间。
3. 执行 **all_reduce**。
4. **sync + stop clock**：同步后停止计时。
5. 用“实际传输字节数 / 耗时”得到聚合带宽。

基准测试的陷阱

GPU kernel 是异步下发的——你调用 **all_reduce** 立刻返回，但计算可能还在排队。所以必须在 **stop clock** 前做 **torch.cuda.synchronize()**，否则你测的是“下发时间”而非“执行时间”。warmup 也是同理：第一次调用要把 kernel 编译/加载进缓存，这一步极慢，不能计入。

传输字节数的计算：张量有 **numel** 个元素，每个元素 **float32** 占 4 字节，故张量字节数为：

$$\text{size} = \text{numel} \times \text{element_bytes} = \text{numel} \times 4$$

聚合带宽（aggregate bandwidth，单位 GB/s）为：

$$B = \frac{\text{size} \times \frac{2(N-1)}{N}}{T_{\text{elapsed}}}$$

¹²视频画面时间区间：00:26:15 – 00:26:45。

4.3 为什么 all-reduce 有 2x 因子

这是本节最微妙、也是被反复追问的一点。

```
206 bandwidth = sent_bytes / total_duration
207 print(f"[all_reduce] Rank {rank}: all_reduce measured bandwidth = {round(bandwidth / 1024**3)} GB/sec", flush=True)
208
209 cleanup()
210
211
212 def reduce_scatter(rank: int, world_size: int, num_elements: int):
213     setup(rank, world_size)
214
215     # Create input and outputs
216     input = torch.randn(world_size, num_elements, device=get_device(rank)) # Each rank has a matrix
217     output = torch.empty(num_elements, device=get_device(rank))
218
219     # Warmup
220     dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
221     if torch.cuda.is_available():
222         torch.cuda.synchronize() # Wait for CUDA kernels to finish
223         dist.barrier() # Wait for all the processes to get here
224
225     # Perform reduce-scatter
226     start_time = time.time()
227     dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
228     if torch.cuda.is_available():
229         torch.cuda.synchronize() # Wait for CUDA kernels to finish
230         dist.barrier() # Wait for all the processes to get here
231     end_time = time.time()
232
233     duration = end_time - start_time
```

Figure 13: reduce-scatter 基准测试：与 all-reduce 对比，没有 2x 因子¹³

2x 因子的来源

- all-reduce = reduce-scatter + all-gather，两步各传输 $\frac{N-1}{N}S$ ，加起来 $\frac{2(N-1)}{N}S$ —— 这是 “2x”。
- reduce-scatter 单独只做一步，传输量是 $\frac{N-1}{N}S$ ，没有 2x。
- all-gather 单独也只有 $\frac{N-1}{N}S$ ，没有 2x。

课堂上的追问澄清了一个易混点：reduce-scatter 名字里虽含 “reduce”，但 NCCL 底层可能用硬件加速把 reduce 和 scatter 融合，实际只算一次传输。Percy 强调：“它叫 reduce-scatter，但其实就是一次 reduction”。于是从字节计数的角度：

$$\text{all-reduce 传输量} = \underbrace{\text{reduce-scatter 传输量}}_{\frac{N-1}{N}S} + \underbrace{\text{all-gather 传输量}}_{\frac{N-1}{N}S} = \frac{2(N-1)}{N}S$$

¹³视频画面时间区间：00:28:30 – 00:29:00。

```

219 # Warmup
220 dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
221 if torch.cuda.is_available():
222     torch.cuda.synchronize() # Wait for CUDA kernels to finish
223     dist.barrier()           # Wait for all the processes to get here
224
225 # Perform reduce-scatter
226 start_time = time.time()
227 dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
228 if torch.cuda.is_available():
229     torch.cuda.synchronize() # Wait for CUDA kernels to finish
230     dist.barrier()           # Wait for all the processes to get here
231 end_time = time.time()
232
233 duration = end_time - start_time
234 print(f"[reduce_scatter] Rank {rank}: reduce_scatter(world_size={world_size}, num_elements={num_elements}) took
(render_duration(duration))", flush=True)
235
236 # Measure the effective bandwidth
237 dist.barrier()
238 data_bytes = output.element_size() * output.numel() # How much data in the output
239 sent_bytes = data_bytes * (world_size - 1) # How much needs to be sent (no 2x here)
240 total_duration = world_size * duration # Total time for transmission
241 bandwidth = sent_bytes / total_duration
242 print(f"[reduce_scatter] Rank {rank}: reduce_scatter measured bandwidth = {round(bandwidth / 1024**3)} GB/sec", flush=True)
243
244 cleanup()

```

Figure 14: 通信成本分析：reduce-scatter 与 all-gather 各 $\frac{N-1}{N}S$ ，all-reduce 是二者之和¹⁴

为什么要费这么大劲算字节？

因为带宽是通信瓶颈的硬约束。知道一次集合通信要搬多少字节、硬件能跑多快，你就能在写代码前预测它会不会成为瓶颈——而不是等训练跑起来才发现卡在通信上。这是 systems 心态的核心：先算账，再动手。

5 数据并行 (Data Parallelism / DDP)

5.1 沿 batch 维切分

数据并行是最直观的并行：模型副本完整地放在每张卡上，把数据按 batch 维切成 N 份，每张卡处理一份。

¹⁴视频画面时间区间：00:31:15 – 00:31:45。

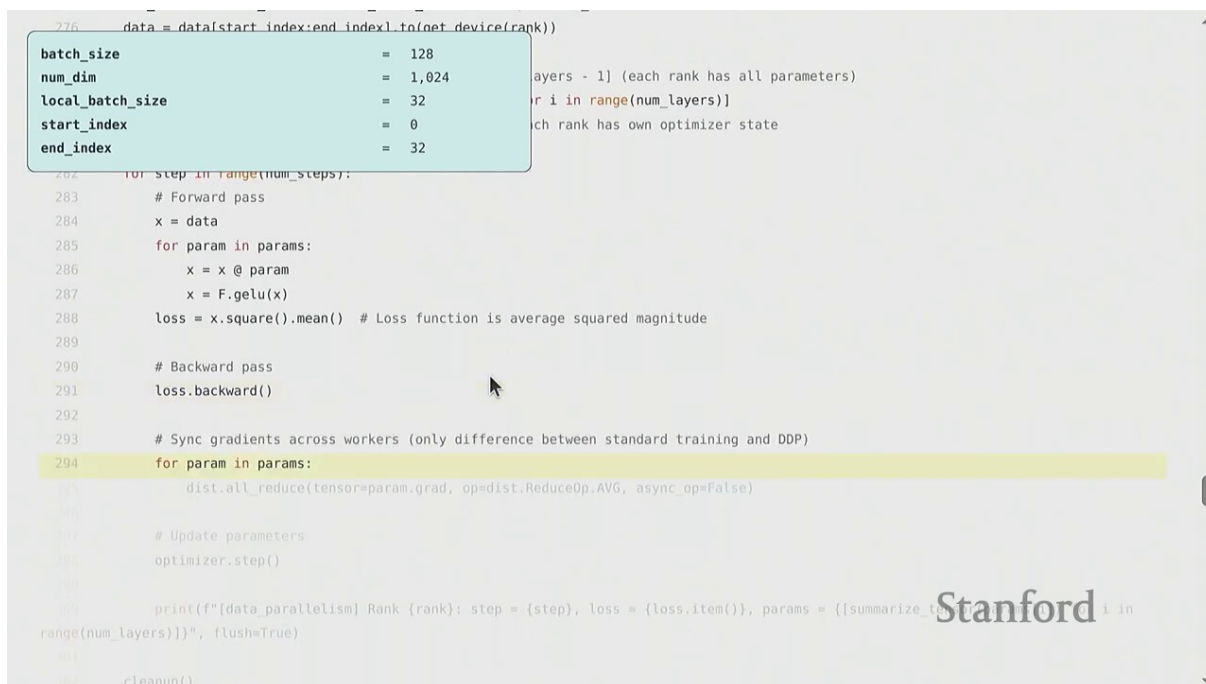


Figure 15: 数据并行：all-reduce 的教科书应用——梯度同步¹⁵

设 batch size 为 B ，hidden dim 为 D ，world size 为 N 。每张卡的本地 batch 为：

$$B_{\text{local}} = \frac{B}{N}$$

每个 rank 根据 rank id 取 batch 中对应的一段：

$$\text{rank } r \text{ 取行 } [r \cdot B_{\text{local}}, (r + 1) \cdot B_{\text{local}})$$

5.2 DDP 的实现：在 SGD 里”注入”一行 all-reduce

Percy 展示了 DDP 的最朴素实现：它几乎和普通 SGD 一模一样，唯一的区别是在反向传播之后、参数更新之前，对每个参数的梯度做一次 all-reduce（取平均）。

```

1 for step in range(num_steps):
2     out = forward(x_local, layers) # □ □ □ □ □ □ □ □ □ □ □ □
3     loss = criterion(out)
4     loss.backward() # □ □ □ □ □ param.grad
5     for param in model.parameters():
6         dist.all_reduce(param.grad, op=dist.ReduceOp.SUM) # □ □ □ □
7         param.grad /= world_size # □ □ □
8     optimizer.step() # □ □ □ □ □ □ □ SGD □ □ □ □ □

```

¹⁵ 视频画面时间区间：00:36:15 – 00:36:45。


```
272 num_dim = data_size[1] # @inspect num_dim
batch_size = 128 # @inspect local_batch_size
num_dim = 1,024 # @inspect start_index
local_batch_size = 32 # @inspect end_index
start_index = 0 # @inspect rank)
end_index = 32

276 # Create MLP parameters params[0], ..., params[num_layers - 1] (each rank has all parameters)
279 params = [get_init_params(num_dim, num_dim, rank) for i in range(num_layers)]
280 optimizer = torch.optim.AdamW(params, lr=1e-3) # Each rank has own optimizer state
281
282 for step in range(num_steps):
283     # Forward pass
284     x = data
285     for param in params:
286         x = x @ param
287         x = F.gelu(x)
288     loss = x.square().mean() # Loss function is average squared magnitude
289
290     # Backward pass
291     loss.backward()
292
293     # Sync gradients across workers (only difference between standard training and DDP)
294     for param in params:
295         dist.all_reduce(tensor=param.grad, op=dist.ReduceOp.AVG, async_op=False)
296
297     # Update parameters
298     optimizer.step()
299
```

Stanford

Figure 16: 数据并行梯度同步：所有 rank 在同一 step 做 all-reduce，是一个同步点¹⁶

5.3 DDP 的关键性质

同步语义

- loss 各卡不同：因为每卡的数据子集不同。
- 梯度 all-reduce 后各卡相同：所以参数更新后各卡也相同。
- 因此 DDP 等价于用 N 倍 batch size 跑一次同步 SGD——只是计算被摊到了 N 张卡上。

all-reduce 是同步点

all-reduce 是一个阻塞式同步点：所有 rank 必须都到达这一步，才能继续。如果某个 rank 漏写了一次 all-reduce（或张量形状不匹配），整个训练会挂起（hang）——没有任何报错，只是永远等待。这是 DDP 调试中最常见的“幽灵 bug”。课堂上有人问“如何保证各 rank 在同一 step”，答案正是：all-reduce 本身就是同步屏障。

作业对应

DDP 是 Assignment 2 的内容（在 Transformer 上实现）。本讲的 MLP 版本是它的最朴素骨架——去掉所有花哨优化，只保留“反向后 all-reduce 梯度”这一行，让机制完全透明。

为什么不在各卡间传优化器状态？

课堂上有一个延伸问题：既然要同步，为什么不干脆把优化器状态（Adam 的一阶/二阶矩）在各卡间搬运同步，而要各卡各自维护？答案在于效率：优化器状态的更新（几个逐元素运算）远比把它在网络上搬运一次要便宜得多。所以正确做法是：每卡各自动维护一份完整优化器状态，只在梯度上做 all-reduce。这是一种典型的“宁可多算一点，也不愿多搬一点”的权衡——计算是廉价的，搬运是昂贵的。

¹⁶视频画面时间区间：00:38:15 – 00:38:45。

6 张量并行 (Tensor Parallelism, Megatron 风格)

6.1 沿 hidden dim 切分模型

数据并行切数据，张量并行则切模型本身：把每一层的参数矩阵沿 hidden 维（宽度）切成 N 块，每张卡持有一块。数据则完整地各卡间流转。

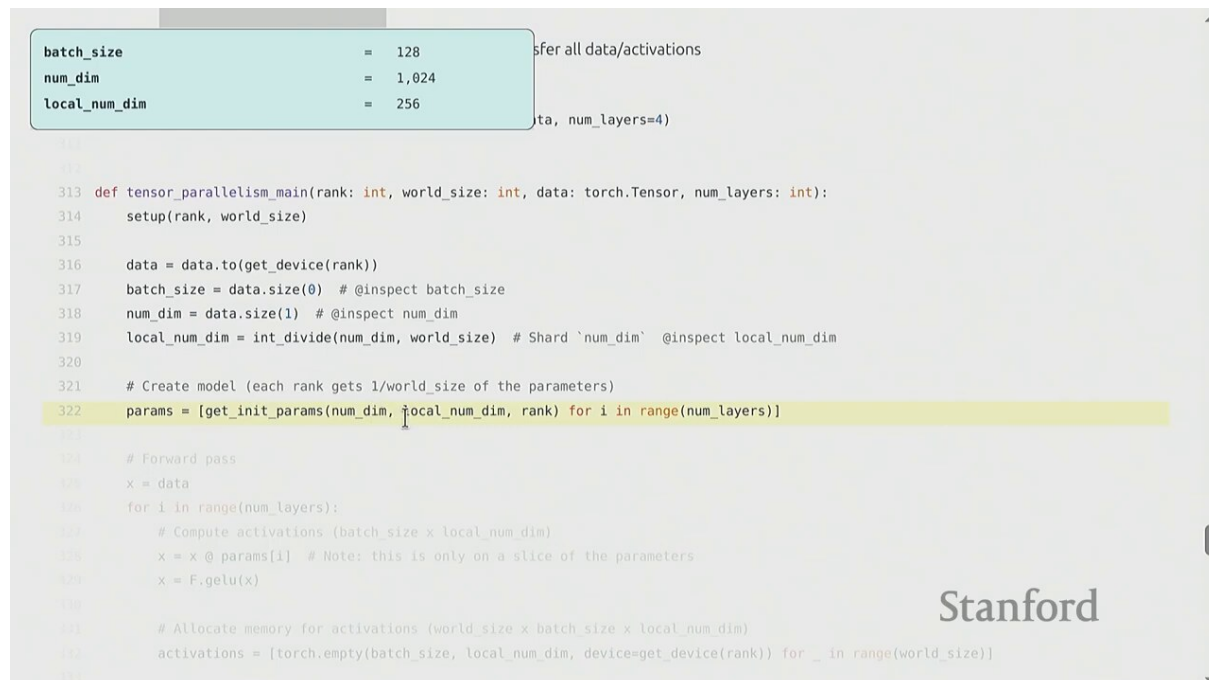


Figure 17: 张量并行：把模型沿 hidden dim 切分， $\text{local_num_dim} = \text{num_dim} / \text{world_size}$ ¹⁷

设 hidden dim 为 D ，world size 为 N ，则每卡的本地维度为：

$$D_{\text{local}} = \frac{D}{N}$$

例如 $D = 1024$ ， $N = 4$ ，则 $D_{\text{local}} = 256$ 。每张卡的参数矩阵形状从 $D \times D$ 变为 $D \times D_{\text{local}}$ ，即每张只持有 $\frac{1}{N}$ 的参数。

为什么要切模型？

因为模型放不进单卡。当模型参数量超过单卡显存时，数据并行（需要每卡存完整模型）就失效了，必须把参数本身分片。这是张量并行存在的根本理由。

6.2 前向传播：all-gather 拼回激活

每张卡只持有 $\frac{1}{N}$ 的参数，所以它算出的激活也只有 $\frac{1}{N}$ （形状为 $B \times D_{\text{local}}$ ）。要让下一层能继续算，必须把所有卡的激活拼回完整维度：

¹⁷视频画面时间区间：00:41:45 – 00:42:15。

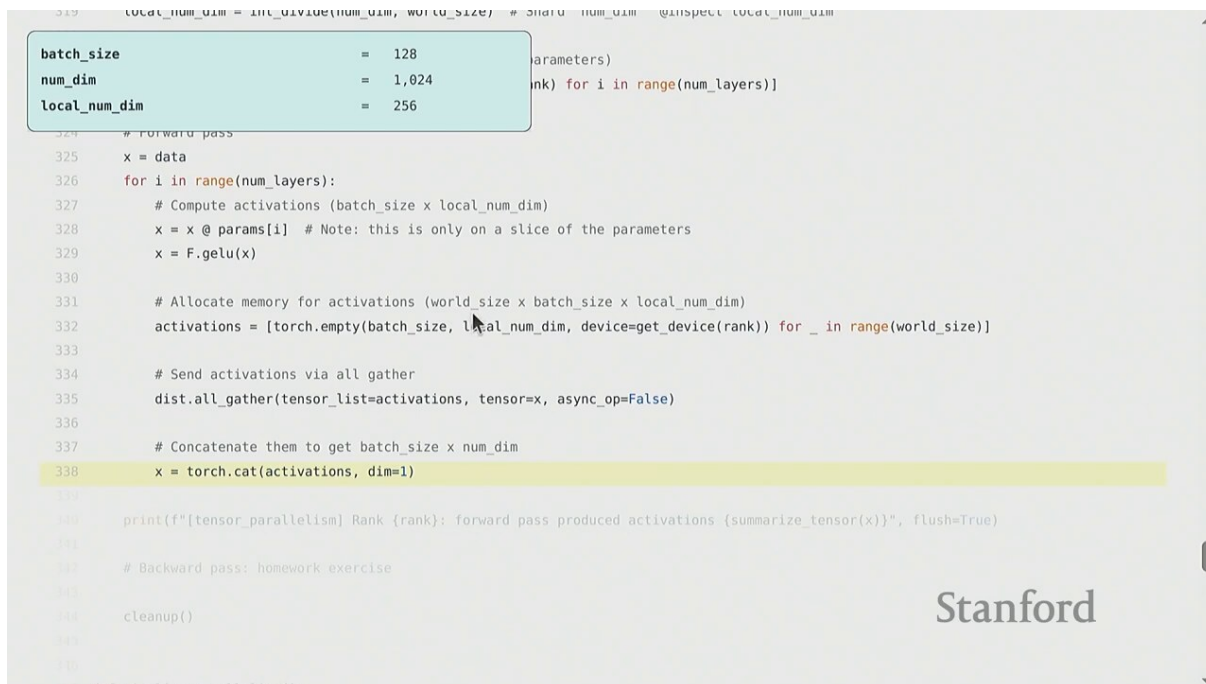


Figure 18: 张量并行前向：每卡算自己那份，再 all-gather 拼回完整输出¹⁸

```

1 for layer in layers:
2     # local x B x D_local
3     x_local = activation(x_local, layer_local) #
4     # world_size B x D_local
5     acts = [empty(B, D_local) for _ in range(world_size)]
6     dist.all_gather(acts, x_local) # x_local
7     x_local = cat(acts, dim=1) # -> B x D

```

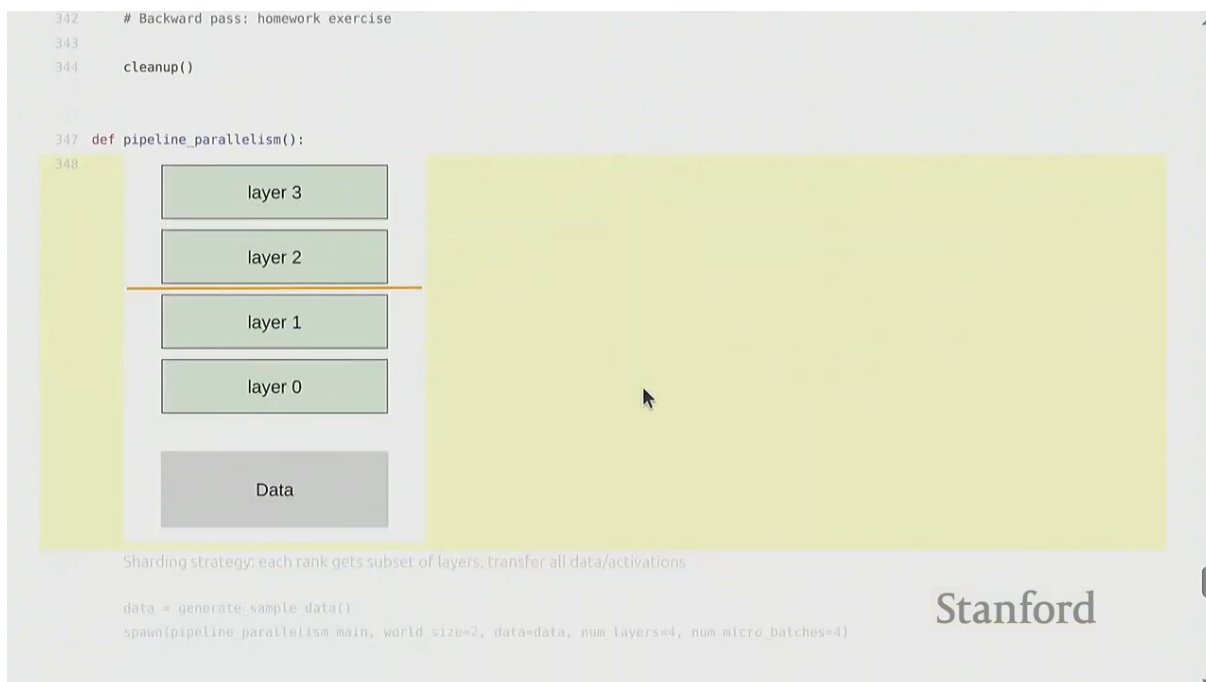


Figure 19: 张量并行中的 all-gather：每个 rank 持局部输出，all-gather 后全员得到完整结果¹⁹

¹⁸ 视频画面时间区间：00:44:45 – 00:45:15。

6.3 通信代价与适用场景

张量并行的通信代价

张量并行每一层之间都要做一次 all-gather，通信极其频繁。因此它对互连带宽要求极高——通常只在节点内（NVLink 全连接，带宽高达 900 GB/s）使用，跨节点走以太网会因通信太慢而得不偿失。

Megatron-LM 的关键洞察

Megatron-LM 论文指出：对 MLP 的两层结构，若把第一层按列切、第二层按行切，中间的 all-gather 可以被一次 reduce-scatter 替代，从而把通信量减半。这是工业级张量并行的核心优化，但本讲的朴素版只演示了最直观的 all-gather 方案。

反向传播被略过

Percy 在课上只演示了张量并行的前向，反向传播“因为时间关系略过”。反向的机制并不难（对 all-gather 求导即 reduce-scatter），但代码 bookkeeping 繁琐。完整的张量并行（含 attention 的 head 维切分、MLP 的列/行切分组合）正是 Megatron-LM 等框架的核心。

7 流水线并行（Pipeline Parallelism）

7.1 按层切分模型

流水线并行把模型沿深度（layers）切：每张卡拿到连续的若干层，数据像流水线一样从第一张卡流向最后一张卡。

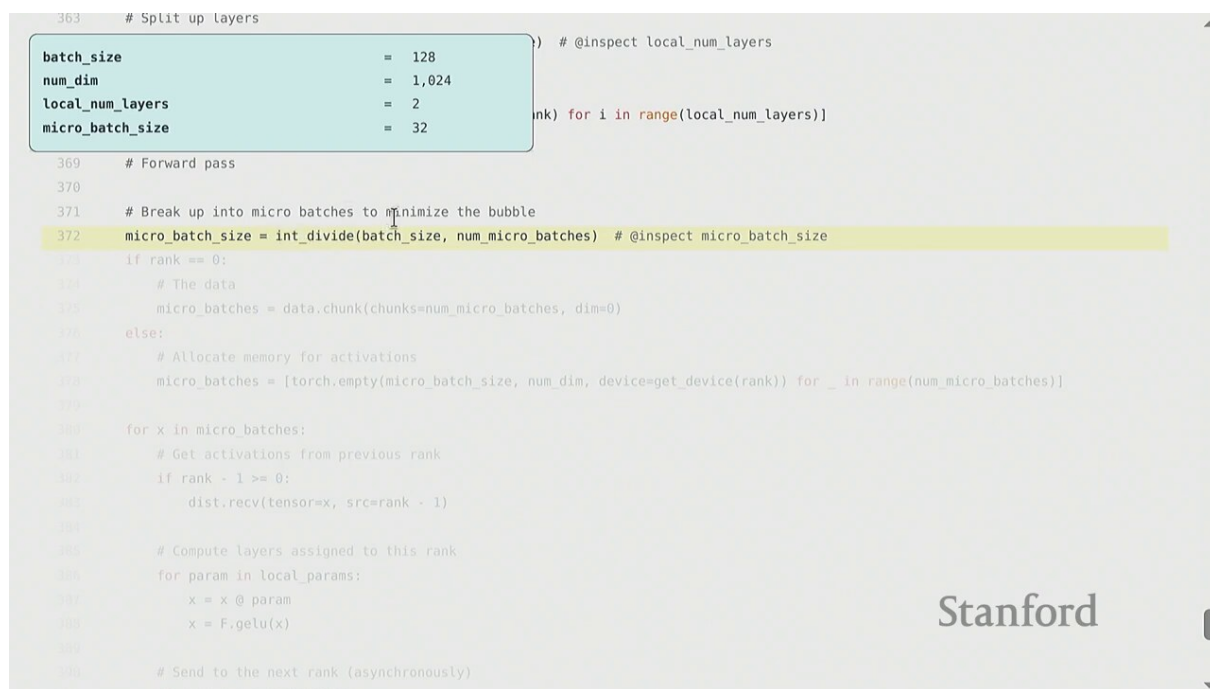


Figure 20: 流水线并行：把模型按层切分，相邻 stage 间用 send/rcv 传递激活²⁰

设模型有 L 层，world size 为 N ，则每张卡分到：

$$L_{\text{per_stage}} = \frac{L}{N}$$

¹⁹ 视频画面时间区间：00:46:15 – 00:46:45。

²⁰ 视频画面时间区间：00:47:45 – 00:48:15。

例如 $L = 4$, $N = 2$, 则 rank 0 持第 0-1 层, rank 1 持第 2-3 层。

7.2 气泡 (bubbles) : 朴素实现的代价

```
374 # The data
batch_size      = 128
num_dim         = 1,024
local_num_layers = 2
micro_batch_size = 32

380 for x in micro_batches:
381     # Get activations from previous rank
382     if rank - 1 >= 0:
383         dist.recv(tensor=x, src=rank - 1)
384
385     # Compute layers assigned to this rank
386     for param in local_params:
387         x = x @ param
388         x = F.gelu(x)
389
390     # Send to the next rank (asynchronously)
391     if rank + 1 < world_size:
392         print(f"[pipeline_parallelism] Rank {rank}: sending {summarize_tensor(x)} to rank {rank + 1}", flush=True)
393         dist.send(tensor=x, dst=rank + 1)
394
395 Not handled: overlapping communication/computation to eliminate pipeline bubbles

# Backward pass: homework exercise
cleanup()
```

Figure 21: 流水线气泡: 朴素实现中 GPU 空闲的“气泡”区域²¹

朴素流水线并行有个致命问题: 气泡。当 rank 0 在处理一个 batch 时, rank 1 只能干等, 直到 rank 0 把结果传过来。这段时间 rank 1 完全空闲, 这就是“气泡”——流水线越深、气泡占比越大, 硬件利用率越低。

microbatch: 缓解气泡的标准手段

把一个 batch 切成 M 个 microbatch, 让它们像流水线上的工件一样依次进入流水线。当 microbatch 1 流到 rank 1 时, microbatch 2 正好可以在 rank 0 上开始算——于是大部分时间所有 rank 都在干活, 气泡被压缩。

设 N 为 stage 数 ($=$ world size), M 为 microbatch 数, 则气泡占比约为:

$$\text{bubble fraction} = \frac{N - 1}{M + N - 1}$$

- $M \gg N$ 时, 气泡占比 $\rightarrow 0$ (流水线被打满)。
- $M = 1$ 时退化为朴素流水线, 气泡最大。

²¹ 视频画面时间区间: 00:49:45 - 00:50:15。

7.3 send / recv: 点对点通信

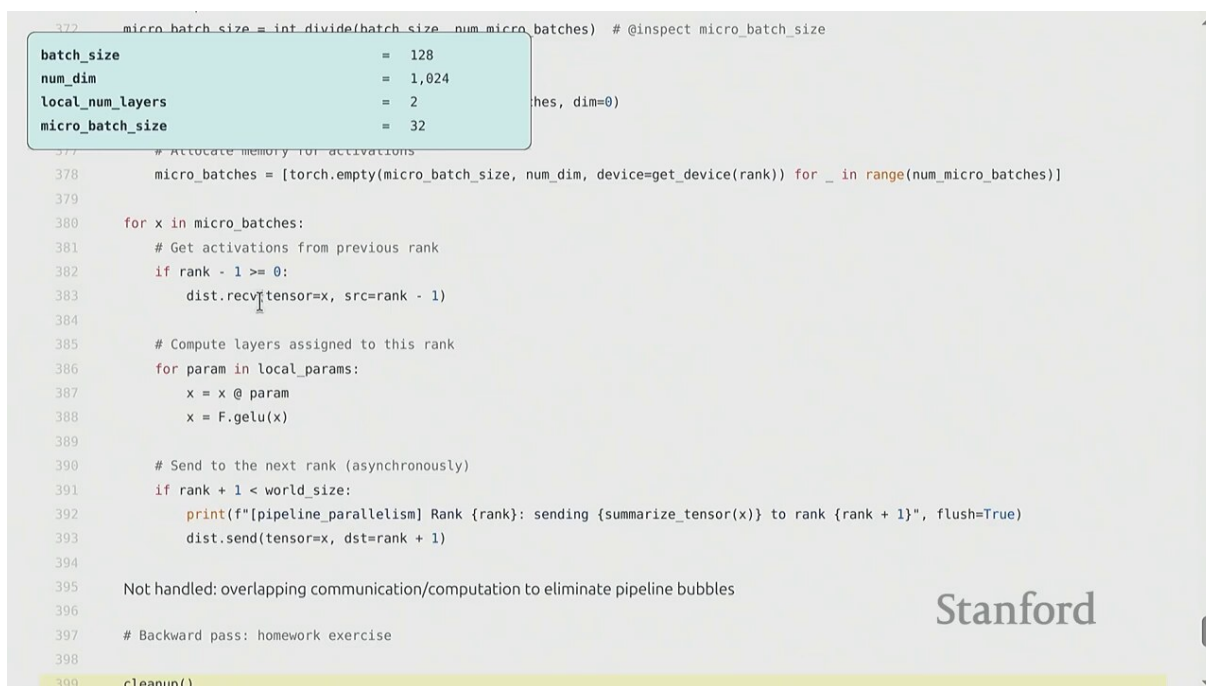


Figure 22: 流水线 send/recv: 相邻 stage 用点对点原语传递激活²²

与数据/张量并行用集合通信不同，流水线并行用点对点原语 send / recv: 每个 rank 只和它的前驱/后继通信。

```
1 for mb in microbatches:
2     if rank > 0:
3         x = recv(src=rank-1)           # □ □ □ □ □
4         x = forward(x, my_layers)      # □ □ □ □ □ □ □
5     if rank < world_size - 1:
6         send(x, dst=rank+1)           # □ □ □ □
```

最后一站与反向

最后一个 rank ($N-1$) 拿到完整的 forward 结果并计算 loss。反向传播则沿流水线逆向：从最后一个 rank 开始算梯度，用 send/recv 把梯度回传给前驱。朴素实现甚至没做反向（课上略过），但完整的流水线并行必须正确交错 forward 与 backward 的调度（如 1F1B、GPipe 等策略）。

²² 视频画面时间区间：00:52:15 – 00:52:45。

7.4 通信与计算的重叠

```
371 # Break up into micro batches to minimize the bubble
372
373 batch_size = 128
374 num_dim = 1,024
375 local_num_layers = 2
376 micro_batch_size = 32
377
378 # Allocate memory for activations
379 micro_batches = [torch.empty(micro_batch_size, num_dim, device=get_device(rank)) for _ in range(num_micro_batches)]
380
381 for x in micro_batches:
382     # Get activations from previous rank
383     if rank - 1 >= 0:
384         dist.recv(tensor=x, src=rank - 1)
385
386     # Compute layers assigned to this rank
387     for param in local_params:
388         x = x @ param
389         x = F.gelu(x)
390
391     # Send to the next rank (asynchronously)
392     if rank + 1 < world_size:
393         print(f"[pipeline_parallelism] Rank {rank}: sending {summarize_tensor(x)} to rank {rank + 1}", flush=True)
394         dist.send(tensor=x, dst=rank + 1)
395
396 Not handled: overlapping communication/computation to eliminate pipeline bubbles
397
398 # Backward pass: homework exercise
399
400 cleanup()
```

Figure 23: 流水线通信-计算重叠：用异步 send/recv 隐藏通信延迟²³

朴素实现里 send/recv 是同步的——发送方会阻塞直到接收方收完。理想做法是把它们改成异步：发送一个 microbatch 的同时，下一层已经开始计算上一个 microbatch。

同步范式 vs 异步范式

现代大规模训练几乎全是同步范式——所有 rank 在 lock step 下工作，梯度严格对齐。10 多年前曾流行过异步训练（asynchronous）：用一个 parameter server，worker 随时上传梯度、随时拉参数，worker 挂了也能容错。但异步会引入梯度陈旧（staleness），在超大规模下反而更难收敛，所以工业界最终回归同步。Percy 特别澄清：他说的“workers 异步运行”只是指它们是不同的进程，但执行逻辑上仍是严格同步的。

²³ 视频画面时间区间：00:55:15 – 00:55:45。

8 3D 并行、FSDP 与声明式分片

8.1 三种并行的组合：3D Parallelism

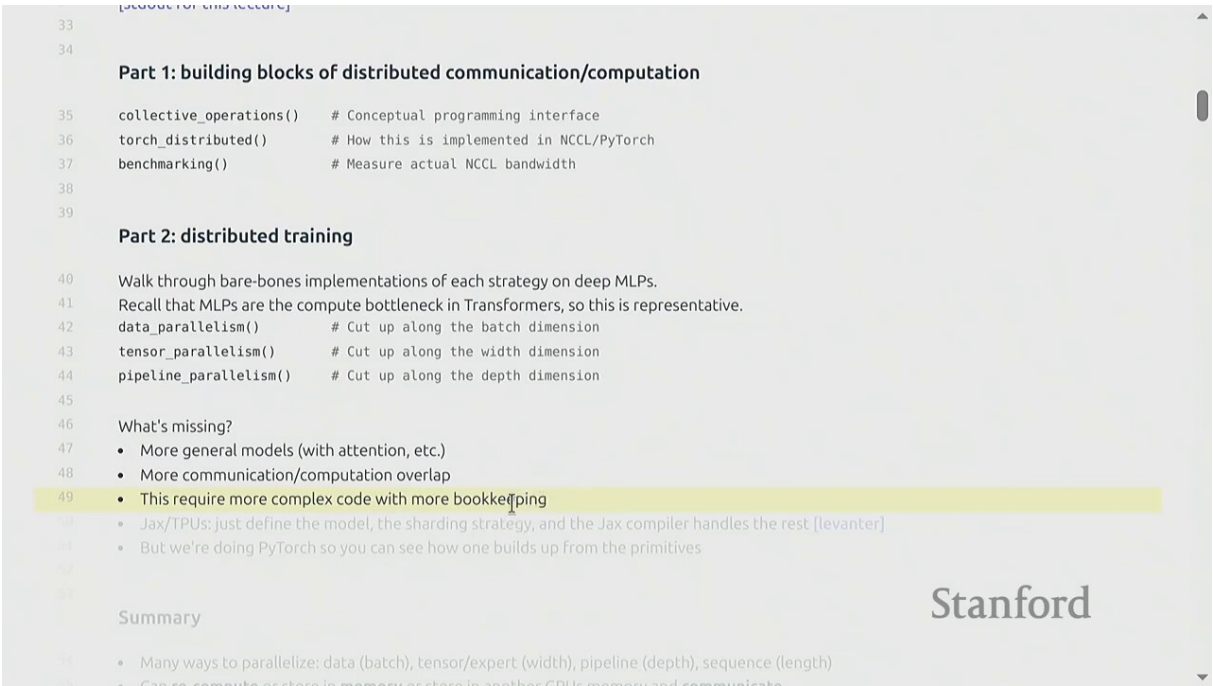


Figure 24: 3D 并行：DP + TP + PP 三种方式各用不同维度，组合使用²⁴

真实的大模型训练从不是单一并行，而是三者组合。每个维度对应一个“轴”，集群被组织成 3D 网格：

并行	切分对象	典型部署位置
TP	模型宽度	节点内（NVLink，带宽极高，否则太慢）
PP	模型深度	节点间或节点内（中等带宽即可，通信少）
DP	数据	节点间（all-reduce 可靠以太网承载）

3D 并行的直觉

把集群分成若干组：组内做 TP（紧密耦合、高带宽），若干 TP 组串成 PP 流水线，再在多条流水线间做 DP（松耦合、all-reduce）。例如训一个超大模型，可能用 8 卡 TP（一个节点）× 4 stage PP × 64 路 DP = 2048 卡。

²⁴ 视频画面时间区间：00:57:45 – 00:58:15。

8.2 FSDP 与 ZeRO: 参数分片

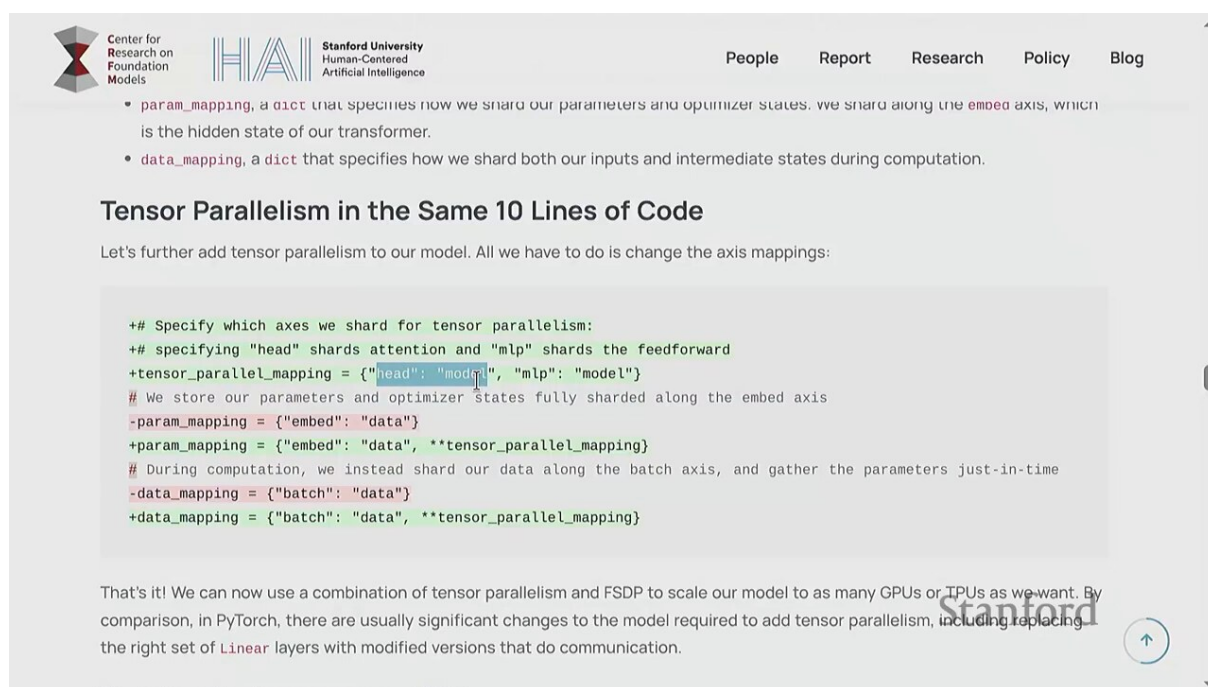


Figure 25: FSDP / 参数分片 (sharding): 声明式定义分片策略, 按需 all-gather 还原参数²⁵

FSDP (Fully Sharded Data Parallel) 是数据并行的进化版: DP 要求每卡存完整参数, 而 FSDP 把参数、梯度、优化器状态都分片到各卡, 只在需要时按需 all-gather 还原。这直接对应 ZeRO 的三个阶段:

ZeRO 阶段	分片对象	显存节省
ZeRO-1	优化器状态	~4x (相对纯 DP)
ZeRO-2	+ 梯度	~8x
ZeRO-3	+ 参数 (= FSDP)	~N 倍 (N 为 GPU 数)

ZeRO-3 / FSDP 的核心机制

前向/反向到某一层时, 先 all-gather 把该层完整参数临时拉到本卡, 算完立即丢弃 (释放显存), 只保留自己的那一片分片。这样任意时刻每卡只持有一层的完整副本 + 所有层的 $\frac{1}{N}$ 分片。代价是通信量增加 (每层都要 all-gather), 但换来的是: 能用更多卡训更大模型。这正是“用额外通信换显存”的经典 trade-off。

8.3 PyTorch 手动实现 vs Jax 声明式分片

Percy 指出, 本讲手写的 MLP 版本是为了让你看清底层 (哪些 all-gather、哪些 send/recv), 但真实工程中没人从零写这些。两条路线:

- **PyTorch 手动路线:** Megatron-LM、PyTorch FSDP。代码“相当 hairy”, 因为要处理任意架构的参数 bookkeeping——哪些层怎么切、shape 怎么对齐、梯度怎么同步。
- **Jax 声明式路线:** 只需声明分片策略 (哪个维度切、映射到哪些 TPU), 编译器自动生成集合通信代码。Stanford 开发的 Lavanter (基于 Jax) 让 FSDP/TP 在 10 行内搞定。

课程为何坚持 PyTorch

Jax 的声明式分片概念上更简洁, 但 CS336 坚持 PyTorch, 因为只有亲手写过 all-gather 和 reduce-scatter, 你才能真正理解通信在发生什么。Lavanter 这类工具是“魔法”, 而本课程的目标正是拆穿魔法。理解了底层, 再用高层工具时你才知道边界在哪。

²⁵ 视频画面时间区间: 00:59:45。

9 总结与延伸

9.1 核心论点

五个核心论点

1. 并行的本质是切维度：DP 切 batch、TP 切宽度、PP 切深度。理解每种方式切哪个维度，其余都是推论。
2. 集合通信是字母表：七个原语（broadcast/scatter/gather/reduce/all-reduce/reduce-scatter/all-gather）组合出所有并行策略。最关键的恒等式是 $\text{all-reduce} = \text{reduce-scatter} + \text{all-gather}$ 。
3. 通信成本可计算：知道一次操作搬多少字节、硬件带宽多少，就能预测瓶颈。all-reduce 的 $2x$ 因子、reduce-scatter 的无 $2x$ ，都来自字节计数。
4. 朴素实现有气泡：流水线并行的气泡可用 microbatch 缓解；现代训练几乎全用同步范式。
5. 3D 并行 + FSDP 是工业现实：TP 在节点内、PP 跨 stage、DP 跨副本；FSDP/ZeRO-3 用分片换显存，用通信换可扩展性。

9.2 课堂问答：硬件演进与持续训练

讲课后半段是开放的问答环节，几个问题切中了大规模训练的长期趋势，值得记录。

专用硬件会取代 GPU 吗？

有听众问：GPU 会不会被 Transformer 专用的硬件取代？Percy 指出，在推理侧，专用硬件（如 Groq、Cerebras）已经相当活跃，它们的核心优势是巨大的片上存储——把尽可能多的权重和激活留在芯片内部，避免昂贵的显存搬运。Cerebras 甚至做到了一个“巨大的 L1 cache”，让你几乎不必把数据搬到片外。GPU 其实背着不少历史包袱（为图形渲染时代的分支计算、临时计算设计），这些在深度学习里并不需要，因此专用硬件有切实的改进空间。但在通用训练上，GPU 短期内仍是主流。

这些技术能用于持续训练吗？

有人问：本讲假设“从头一次性训练”，这些并行技术能否用于增量/持续训练（新数据来了不重训，而是增量补全）？Percy 的回答很干脆：完全可以。这些并行策略的工作单元就是“梯度步”，你只要从一个已有的 checkpoint 恢复，就能继续做梯度下降——没有任何东西要求你必须从零开始。持续训练在工程上与从头训练共享同一套并行基础设施，区别只在于数据流和学习率调度。

Checkpoint 的位置选择

另一个细节问题：流水线/张量并行中，激活该在哪些位置存 checkpoint（以省显存）？Percy 的经验法则是“每隔几层存一次”，尤其要选在那些容易重新计算的边界——比如 MLP 之后再接一个逐点线性层，这种“两步几乎等价于一步”的地方，没必要存两份激活，存一份即可。这正是“用额外计算换显存”思想在并行场景下的具体应用。

9.3 三种并行的对比总结

维度	DP	TP	PP	FSDP
切分对象	数据	模型宽度	模型深度	参数 + 梯度 + 优化器
通信原语	all-reduce	all-gather	send/recv	all-gather + reduce-scatter
通信频率	每步一次	每层一次	每 stage 一次	每层按需
互连要求	中（以太网可）	极高（NVLink）	中低	中高
显存节省	无	中（参数分片）	中（层分片）	最大（全分片）

9.4 下节预告与配套阅读

接下来将进入 systems 模块的更深层，或转向 scaling laws / data 等模块。以下是建议按进度阅读的配套论文：

论文/资源	对应知识点
Megatron-LM (Shoeybi et al.)	张量并行的列/行切分优化
ZeRO: Optimizing 500B Training (Rajbhandari et al.)	ZeRO-1/2/3 与 FSDP
PyTorch FSDP 文档	参数分片、按需 all-gather
GPipe / PipeDream	流水线并行调度、气泡分析
NCCL 文档	ring all-reduce 的底层实现
Lavander (Jax)	声明式分片的对比视角